



UNIVERSITY
OF COLOGNE

WORKSHOP 08

Advanced Analytics and Applications

Agenda

1

Lecture Recap

2

Exemplary Exam Questions

3

Coding Tasks

Multi-Armed Bandits (MAB)

- MAB are special cases of decision problems with a single state, where the agent is faced with the same choice among different actions again and again and can only learn how good or bad certain actions are.
- At the beginning (playing for the first time), we have no expectations of the true values $q_*(a)$ of actions a , thus we need to try actions and update our expectations accordingly after trying the action n times:

$$Q(a) = Q(a) + \frac{1}{n}[R - Q(a)]$$

- We hope that $Q(a) \rightarrow q_*(a)$ as $n \rightarrow \infty$ because that could then be our policy:
Greedy select the value-maximizing action all the time.
- In reality, however, we have to **explore** other actions and balance that with **exploiting** well-known actions:

```
A simple bandit algorithm

Initialize, for  $a = 1$  to  $k$ :
   $Q(a) \leftarrow 0$ 
   $N(a) \leftarrow 0$ 

Loop forever:
   $A \leftarrow \begin{cases} \arg \max_a Q(a) & \text{with probability } 1 - \varepsilon \quad (\text{breaking ties randomly}) \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$ 
   $R \leftarrow \text{bandit}(A)$ 
   $N(A) \leftarrow N(A) + 1$ 
   $Q(A) \leftarrow Q(A) + \frac{1}{N(A)}[R - Q(A)]$ 
```

Non-Stationary MAB Problems

- Up until now we assumed that $Q(a) \rightarrow q_*(a)$ as $n \rightarrow \infty$ which means that the value of an action does not change over time (it's stationary).

- If that changes, however, it's more important to consider recent rewards. With

$$Q(a) = Q(a) + \frac{1}{n} [R - Q(a)]$$

the opposite happens, though: Recent rewards are considered less and less (because of the step size $\frac{1}{n}$)

- To combat, we can introduce a constant step size parameter $\alpha \in (0,1]$ that we either use constant

$$Q_{n+1} = Q_n + \alpha [R_n - Q_n]$$

or as a decaying weight on older rewards

$$Q_{n+1} = (1 - \alpha)^n Q_n + \sum_{i=1}^n \alpha (1 - \alpha)^{n-i} R_i$$

Advanced Techniques

- There are a couple of problems with both stationary and non-stationary bandit algorithms, and there are different approaches to combat them:
 1. At the start, we set $Q \rightarrow 0$ which presents a cold start problem and an initial bias that we need to overcome: **Optimistic Initial Values** try to alleviate that by setting $Q \rightarrow X$ where $X > 0$ to force more exploration through exploitation of artificially high Q .
 2. When we explore (non-greedy action), how do we choose among these?
Upper-Confidence-Bound action selection bases a preference on the *potential* for being optimal, which includes the current knowledge ($Q_t(a)$) and *uncertainty*:

$$A_t = \underset{a}{\operatorname{argmax}} \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

Advanced Techniques

What we disregarded up until now: The alternatives.

This where the **Gradient Bandit Algorithm** comes in with a baseline that dictates probabilistic choice of actions based on their expected return over the baseline (average reward up until now):

$$\Pr\{A_t = a\} = \frac{e^{H_t(a)}}{\sum_{b=1}^k e^{H_t(b)}} = \pi_t(a)$$

And we update similarly to *stochastic gradient descent*, for the chosen action:

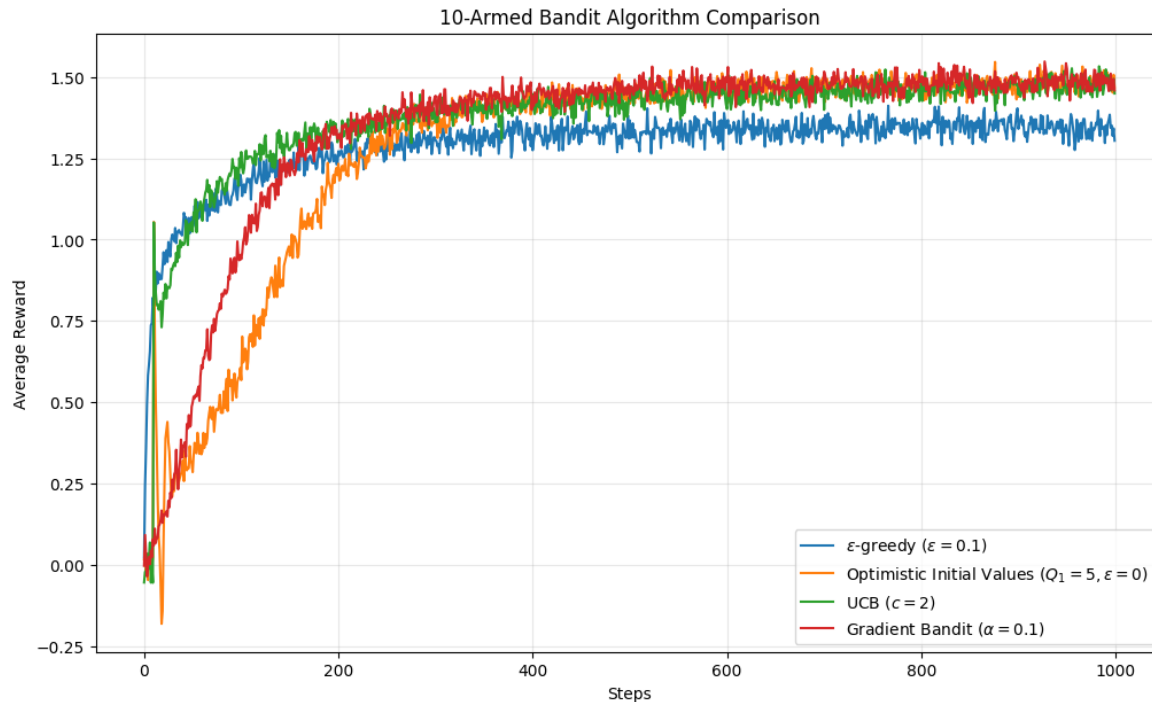
$$H_{t+1}(A_t) = H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t))$$

for the non-chosen actions $a \neq A_t$:

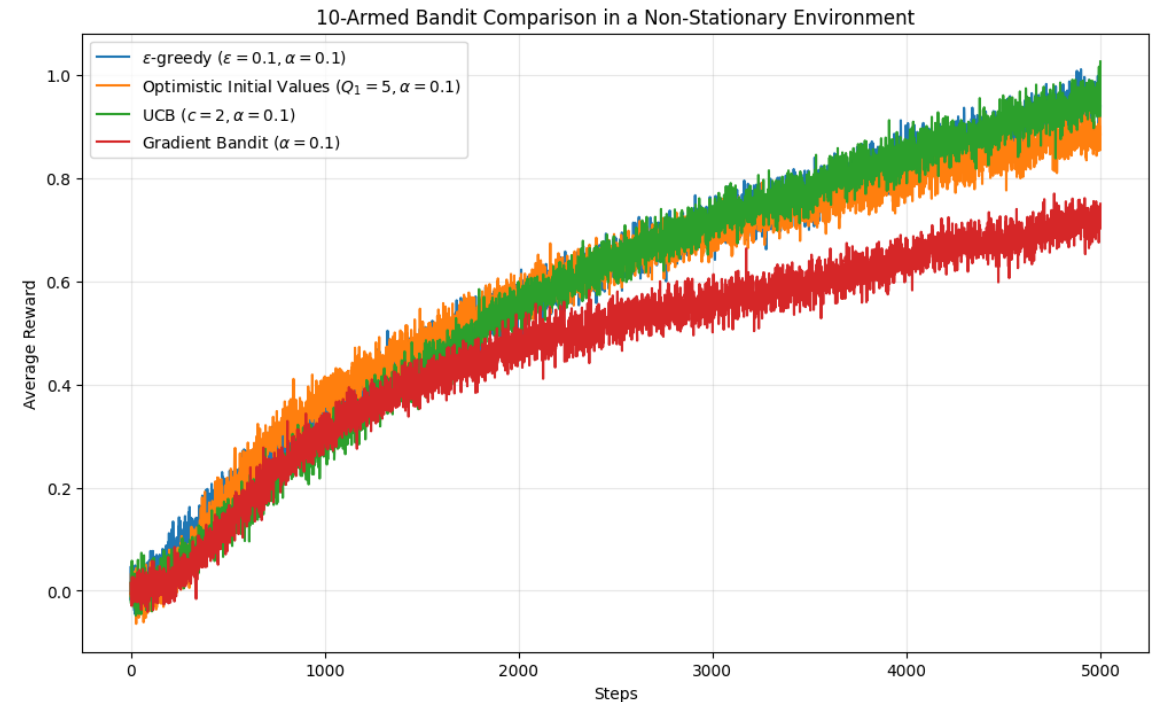
$$H_{t+1}(a) = H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a)$$

Advanced Techniques: Comparison

Stationary



Non-Stationary



Based on the example 10-lever testbed from Sutton & Barto: <http://incompleteideas.net/book/code/code2nd.html>, Chapter 2

Agenda

1

Lecture Recap

2

Exemplary Exam Questions

3

Coding Tasks

Question 1: Gradient Bandit in Non-Stationary Problems

Question: We saw that gradient bandit algorithms can perform badly in non-stationary problems. Explain why.

Answer: Because the updates are based on the error from the baseline and the baseline tracks all historical rewards and averages them. When the problem is non-stationary and actions will prompt different rewards, that baseline is not a good estimation of the expected average reward at any t .

One solution can be to decay rewards in the history.

Question 2: UCB in Non-Stationary Problems

Question: We saw that UCB can perform really well in non-stationary problems. Explain why.

Answer: Because in an environment where frequent re-evaluation of different options (actions) is essential (due to the non-stationary) the **uncertainty term forces to periodically revisit neglected actions**:

$$+c \sqrt{\frac{\ln t}{N_t(a)}}$$

For actions that are frequently chosen $N_t(a)$ is large and the uncertainty term grows smaller. At the same time, the $N_t(a)$ of all other actions don't move, which let's their uncertainty term grow large enough to (after some time) force a visit. In combination with the static step size α we decay old rewards and quickly realize that the newly visited action might have changed to now be better.

Agenda

1

Lecture Recap

2

Exemplary Exam Questions

3

Coding Tasks

Contact



For general questions and enquiries on **research, teaching, job openings and new projects** refer to our website at www.is3.uni-koeln.de



For specific **enquiries regarding this course** contact us by sending an email to the **IS3 teaching address** at is3-teaching@wiso.uni-koeln.de

To help us process your request efficiently, please use the following subject line format:

[AAA] <Your request subject>